00 01 02 03 04 05 06 07 08 09 0A 0B **0C** 0D

ALGORİTMA VE PROGRAMLAMA – II

HAFTA 13

Dinamik Programlama

Örtüşen Alt Problemlerden Tablolara

MODÜL V · OPTİMİZASYON VE SENTEZ

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** **Dinamik Programlama**	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

Geçen haftadan dört soru (yanıtlar bir sonraki slaytta)

- **S1 (H12).** {1, 3, 4} sisteminde 1 … 100 tutarlarının kaçında açgözlü para üstü en az parayı verir? Yanıldığı en küçük tutar kaç?
- **S2 (H3).** Saf özyinelemeli fib(5) kaç çağrı yapar? Kaçı daha önce çözülmüş bir alt problemin tekrarıdır?
- **S3 (H12).** Kapasite 50: A(10 kg, 60), B(20 kg, 100), C(30 kg, 120). Yükler bölünemiyorsa birim değere göre açgözlü kaç değer toplar, en iyi kaç?
- **S4 (H11).** Bir DAG'ın topolojik sırası her kenar u → v için neyi güvence eder?

Geçen haftadan: yanıtlar

- **S1:** 76'da en iyi; ilk yanıldığı tutar **6** (H12 Örnek 1): açgözlü 4 + 1 + 1, en iyisi 3 + 3.
- **S2:** 15 çağrı, yalnız 6 farklı alt problem; **9** çağrı tekrar (H3 Örnek 1) — DP bu tekrarları atar.
- **S3:** açgözlü AB → **160**, en iyi BC → **220** (H12 Örnek 4): 0/1 sürümde açgözlü seçim özelliği yok.
- **S4:** her kenar u → v'de u, v'den önce gelir (H11 Örnek 11: 1 2 4 3 5 6 7 0 8). DP'de “bağımlı olunan önce” kuralı tam budur.

HAFTANIN BÜYÜK SORUSU

Büyük Soru: bütün olasılıkları denemeden en iyiyi bulmak

3 → 2

{1, 3, 4} ile 6: açgözlü 4 + 1 + 1, DP 3 + 3

242 785 → 49

fib(25): saf özyineleme ↔ memoization (çağrı)

111 → 117

12 ders, AKTS ≤ 30: oran açgözlüsü ↔ DP (tercih puanı)

- Açgözlü yöntemin başarısız olduğu bir bozuk para sisteminde (H12), tüm olasılıkları denemeden en iyi yanıtı nasıl buluruz?

GERÇEK DÜNYADAN

1952 · 1970 · 1974 · 1976

“Dinamik programlama”: bir adın öyküsü ve bugünkü izleri

- **Bellman'ın (1984) anlattığına göre** ad 1950 sonbaharında RAND'da seçildi: Savunma Bakanı Wilson “araştırma” sözcüğünden nefret ediyordu; “dynamic” kötü anlamda kullanılamazdı, “bir Kongre üyesinin bile itiraz edemeyeceği” bir ad.
- **Kronoloji notu:** Wilson göreve 28 Ocak 1953'te başladı; terim bir makale başlığında Ağustos 1952'de geçti (Bellman, 1952). Anlatı Bellman'ın kendi hatırasıdır.
- **Needleman ve Wunsch (1970):** iki proteinin amino asit dizilerindeki benzerlikleri aramak için genel bir yöntem — dizi hizalama.
- **Wagner ve Fischer (1974):** düzenleme uzaklığı, yazım düzeltme. **Hunt ve McIlroy (1976):** `diff`'in merkezinde LCS; doğrudan DP O(mn) zaman ve bellek.

Bellman (1952, 1984); Dreyfus (2002); OSD Historical Office (t.y.); Needleman & Wunsch (1970); Wagner & Fischer (1974); Hunt & McIlroy (1976).

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 13.1	Bir problemin dinamik programlamaya uygunluğunu örtüşen alt problemler ve en iyi alt yapı ölçütleriyle belirler.
ÖÇ 13.2	Özyinelemeli bir çözümü memoization ile yukarıdan aşağıya, tablolamayla aşağıdan yukarıya dinamik programlamaya dönüştürür.
ÖÇ 13.3	Durum tanımı, geçiş bağıntısı, temel durumlar ve hesaplama sırasını açıkça yazarak klasik DP problemlerini (bozuk para, LIS, LCS, düzenleme uzaklığı, 0/1 sırt çantası) çözer.
ÖÇ 13.4	DP tablosundan en iyi çözümün kendisini geri çıkarır (reconstruction); bellek kullanımını satır tamponlarıyla azaltır.
ÖÇ 13.5	Kaba kuvvet, geri izleme, açgözlü ve DP çözümlerini aynı problem üzerinde doğruluk ve karmaşıklık açısından karşılaştırır.

TAHMİN ET

fib_memo(6): memo başta −1, her F(i) ilk hesaplandığında nota yazılır, sonraki çağrılar nottan döner. Toplam kaç çağrı yapılır (ilk çağrı dahil)?

A 7: her F(i) bir kez

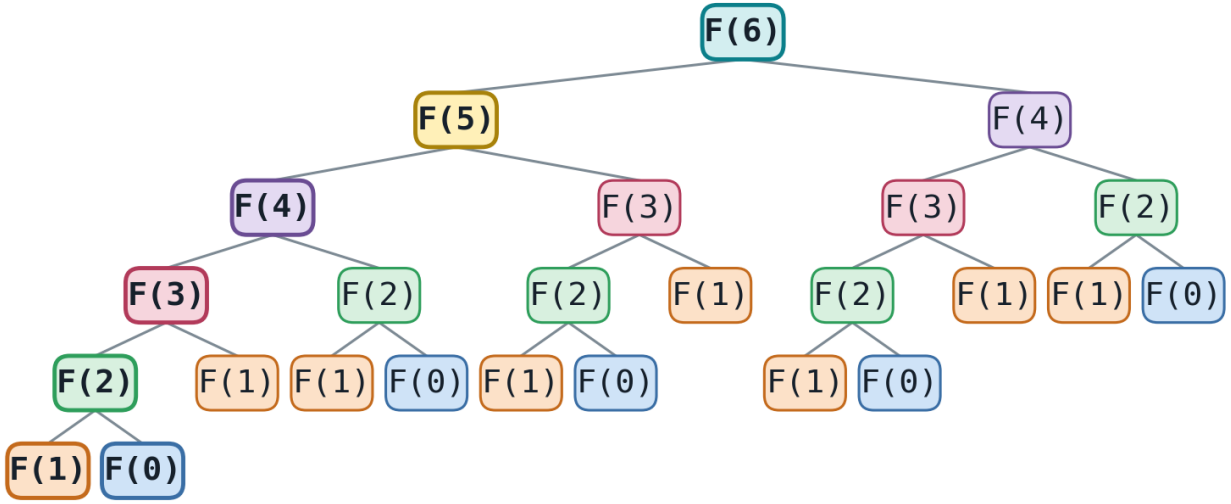
B 11

C 13

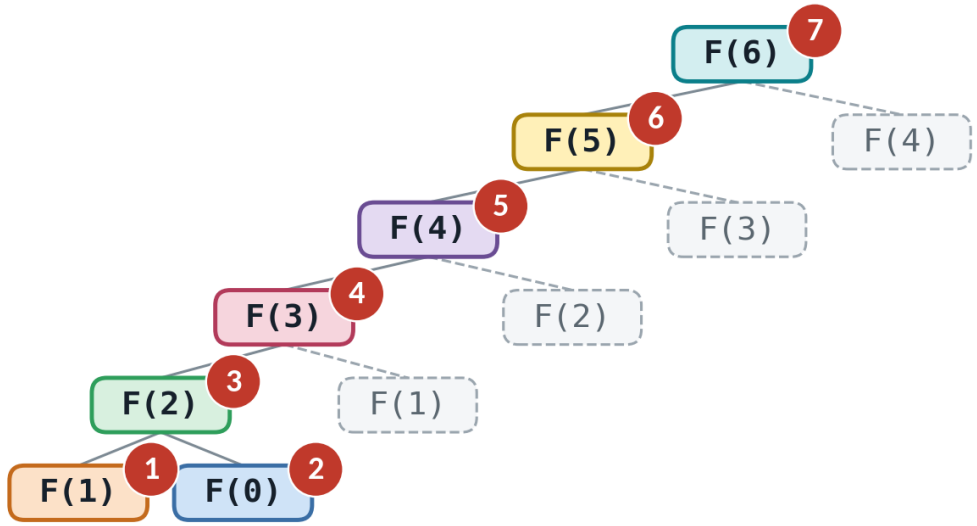
D 25: saf özyinelemeyle aynı

Fibonacci'ye dönüş: aynı alt problemler tekrar tekrar

Saf: 25 çağrı



Memo: 11 çağrı, 4 isabet



fib_saf(6): 25 çağrı, 7 farklı alt problem · fib_memo(6): 11 çağrı, rozet: memo'ya yazılış sırası 1 0 2 3 4 5 6

ALGORİTMA KİMLİK KARTI

Fibonacci: memoization, tablolama, iki değişken *memoization, tabulation*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
DP: F(i − 1), F(i − 2) bir kez çözülür (memo ya da tablo)	n ≥ 0; uint64_t'ye F(93)'e kadar	Θ(n)	Θ(n)	Θ(n)	O(n) / O(1)

Özellikler	n = 25: saf 242 785 çağrı · memo 49 çağrı · tablo 26 hücre · iki değişken O(1)
Değişmez / doğruluk	tablo[0..i] = F(0..i); iki değişkende (onceki, simdiki) = (F(i − 1), F(i))
Kütüphane karşılığı	C: yok · C++: yok · Python: `functools.cache`
Ne zaman kullanılır	Doğrusal yinelemeli bağıntılar: merdiven, döşeme, sayma
Ne zaman kullanılmaz	Tek ve çok büyük n (matris üssü O(log n)); saf özyineleme asla

Üç dilde: tablonun iki değişkene inmesi

H13_01_fibonacci.c — C

```c
for (int i = 2; i <= n; i++) {
    long long sonraki = onceki + simdiki;
    onceki = simdiki;
    simdiki = sonraki;
}
return simdiki;
```

H13_01_fibonacci.cpp — C++

```cpp
for (int i = 2; i <= n; i++) {
    long long sonraki = onceki + simdiki;
    onceki = simdiki;
    simdiki = sonraki;
}
return simdiki;
```

H13_01_fibonacci.py — Python

```python
for _ in range(2, n + 1):
    onceki, simdiki = simdiki, onceki + simdiki
return simdiki
```

Yalnız son iki değer tutulur: O(1) bellek; Python'da demet ataması `onceki, simdiki = simdiki, onceki + simdiki` (satırlar: C 73–78 · C++ 73–78 · Python 59–61)

DP'nin iki koşulu

Örtüşen alt problemler

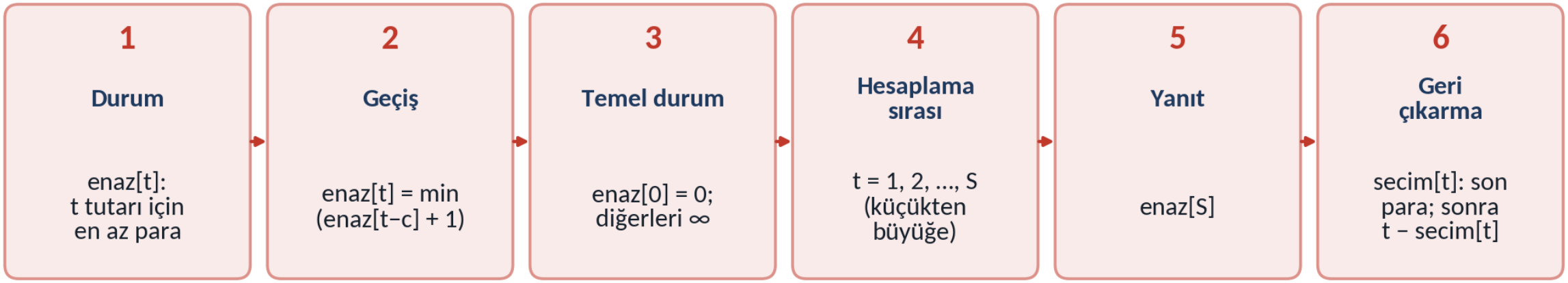
- Özyinelemeli çözüm **aynı** alt problemleri tekrar ister: F(n − 1) de F(n − 2)'yi çağırır; 6 tutarı 5, 3, 2'yi; 5 de 4, 2, 1'i.
- Farklı alt problem az (n + 1, S + 1), çağrı üstel.
- **Böl ve fethetten farkı (H6):** birleştirmeli sıralamanın alt problemleri ayrıktır; not almak kazandırmaz.

En iyi alt yapı

- En iyi çözüm, alt problemlerin **en iyi** çözümlerinden kurulur: 6'nın en iyi ödemesinin son parası c ise kalanı 6 − c'nin en iyi ödemesidir.
- Kanıt: “kes ve yapıştır” — daha kötü bir alt çözüm iyileştirilebilirdi.
- **Açgözlüden farkı (H12):** açgözlü tek seçeneği dener; DP hepsini dener ama her alt problemi bir kez çözer.

DP reçetesi: kod yazmadan önce altı satır

DP reçetesi — örnek: bozuk para (en az para, Örnek 3)



t	0	1	2	3	4	5	6
enaz[t]	0	1	2	1	1	2	**2**
secim[t]	–	1	1	3	4	1	**3**

6 → 3 → 0:
3 + 3 = 2 para

Durum · geçiş · temel durum · hesaplama sırası · yanıt · geri çıkarma — örnek: bozuk para {1, 3, 4}, S = 6 (Örnek 3)

İlk durum–geçiş tanımı: merdiven (1 ya da 2 basamak)

```
algoritma MERDİVEN(n)   ▷ yol[i]: i. basamağa kaç yolla çıkılır
  yol[0] ← 1;  yol[1] ← 1                          ▷ temel durum
  i ← 2'den n'ye:  yol[i] ← yol[i – 1] + yol[i – 2]   ▷ son adım 1 ya da 2
  döndür yol[n]
```

c/H13_02_merdiven.c · satır 26–33

C

```c
static void merdiven_tablo(int n)
{
    yol[0] = 1;                          /* temel durum */
    yol[1] = 1;
    for (int i = 2; i <= n; i++) {
        yol[i] = yol[i - 1] + yol[i - 2]; /* son adım 1 ya da 2 */
    }
}
```

Örnek 2: n = 0 … 40 için yol[n] = F(n + 1) (program denetler); yol[40] = 165 580 141 · {1, 2, 3}: üçüncü terim eklenir

Yukarıdan aşağıya (memo) ↔ aşağıdan yukarıya (tablo)

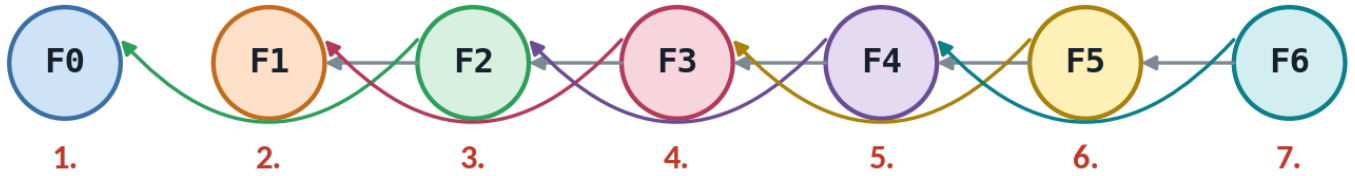
Kavram 6, alt küme toplamı (n = 30): dizi notu 8 835 hücre ayırır, memo yalnız **6 005** tanesini (%68,0) ziyaret eder.

	Memoization	Tablolama
Yazım	özyinelemeli tanım + not tablosu	döngüler; doldurma sırası elle seçilir
Hesaplanan durumlar	yalnız gerekenler	tablonun tamamı
Çağrı yığını	derinlik kadar (Python: sınır)	yok
Bellek azaltma	zor	kolay: satır tamponu, iki değişken
Ne zaman?	seyrek durum uzayı; doğal özyineleme	yoğun durum uzayı; derin zincir

- **Not tutmanın iki aracı:** dizi (−1 = hesaplanmadı) ↔ karma tablosu (H10): K06'da karma notu yalnız ziyaret edilen 6 005 durumu saklar.
- **Kadane (H2) de bir DP'dir:** biten[i] = max(a[i], biten[i − 1] + a[i]); yalnız son hücre gerekir → tek değişken.

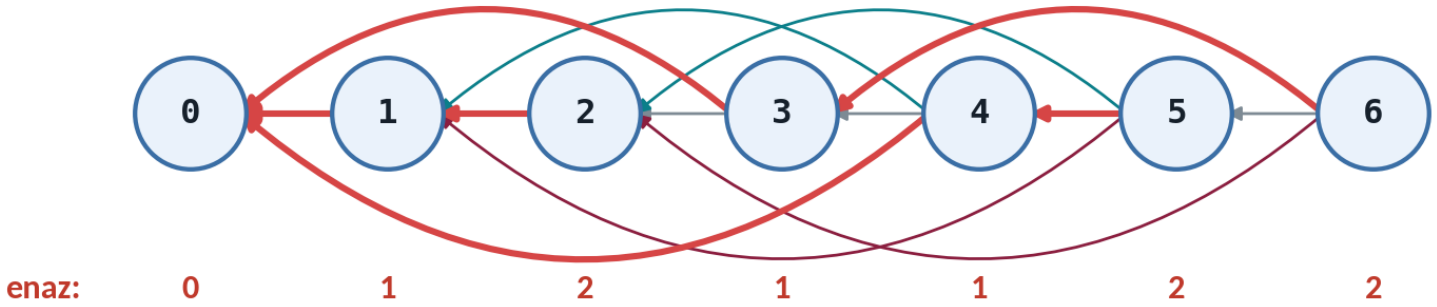
Alt problem DAG'ı: hesaplama sırası ve yollar

(a) Fibonacci, n = 6: F(i) ← F(i−1), F(i−2)



alttaki sayılar: hesaplama sırası (topolojik sıra: bağımlı olunan önce)

(b) Bozuk para {1, 3, 4}, S = 6: enaz[t] = 0'dan t'ye en kısa yol (kenar = bir para)



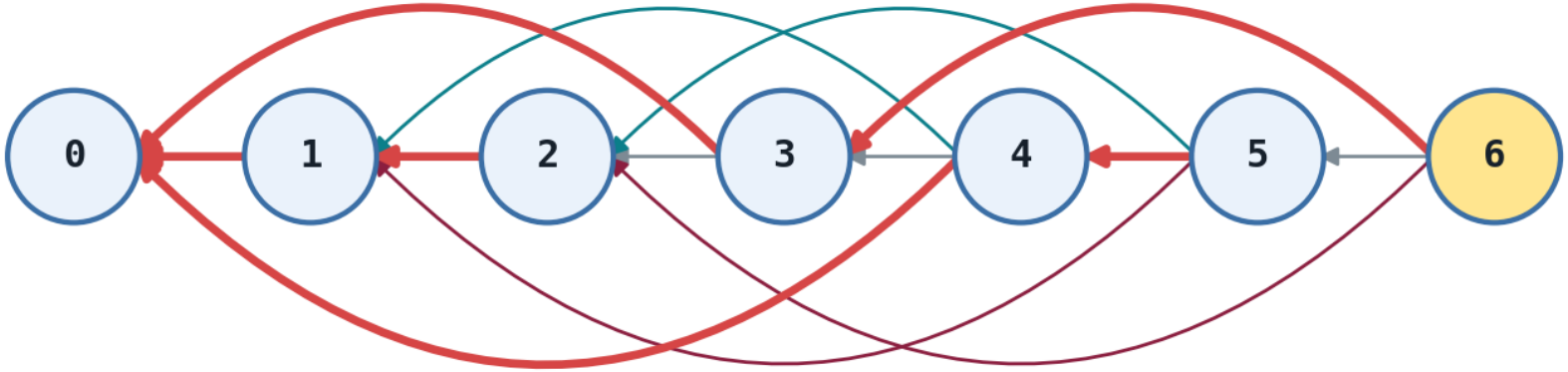
gri: c = 1 · turkuaz (üstte): c = 3 · bordo (altta): c = 4 · kırmızı kalın: secim[t]

- Düğüm = alt problem, kenar = "sonucunu kullanır"; graf **döngüsüz**.
- Hesaplama sırası bir **topolojik sıradır** (H11).
- enaz[t], 0'dan t'ye **en kısa yol**: Kavram 7'de 13 kenar, 7 başarılı gevşetme (H12).
- LIS: uz[i] = i'de biten **en uzun yol** (19 kenar, 4 düğüm).
- Sayma DP'si (merdiven, Örnek 4) yolları **sayar**; öbek gerekmez, negatif kenar sorun değil.

Bozuk para {1, 3, 4}, S = 6: durum–geçiş diyagramı

Durum–geçiş diyagramı: paralar {1, 3, 4}, S = 6 (düğüm t, ok t → t − c)

gri: c = 1 · turkuaz (üstte): c = 3 · bordo (altta): c = 4 · kırmızı kalın: secim[t]



t	0	1	2	3	4	5	6
adaylar*	–	1:1	1:2	1:3 3:1	1:2 3:2 4:1	1:2 3:3 4:2	1:3 3:2 4:3
enaz[t]	0	1	2	1	1	2	2
secim[t]	–	1	1	3	4	1	3

* adaylar c:v — para c ile enaz[t − c] + 1 = v; en küçük v seçilir (eşitlikte küçük c)

TAHMİN ET

Önceki slayttaki tabloyu bir sütun uzatın: t = 7'de adaylar c : enaz[7 − c] + 1. Paralar artan sırayla denenip yalnız kesin küçükte güncellenirse secim[7] kaçtır?

A 3: 7 = 3 + 4

B 1: 7 = 1 + 3 + 3

C 4: 7 = 4 + 3

D 2: iki para gerekir

Bozuk para: en az para ve geri çıkarma

```
algoritma EN-AZ-PARA(para[1..n], S)   ▷ enaz[0] = 0, diğerleri ∞
  t ← 1'den S'ye, her c ∈ para için:
    eğer c ≤ t ve enaz[t − c] ≠ ∞ ve enaz[t − c] + 1 < enaz[t] ise
      enaz[t] ← enaz[t − c] + 1;  secim[t] ← c
  döndür enaz[S]                          ▷ ∞: ödenemez
GERİ-ÇIKAR(t):  iken t > 0:  yaz secim[t];  t ← t − secim[t]
```

c/H13_03_bozuk_para_en_az.c · satır 36–43

C

```c
for (int i = 0; i < k; i++) {
    int c = paralar[i];
    if (c <= t && enaz[t - c] != SONSUZ &&    /* taşma koruması */
        enaz[t - c] + 1 < enaz[t]) {
        enaz[t] = enaz[t - c] + 1;
        secim[t] = c;
    }
}
```

Θ(n·S) zaman, Θ(S) bellek · t = 1 … 20 arasında açgözlü 4 tutarda DP'den fazla para verir (Örnek 3)

ALGORİTMA KİMLİK KARTI

Bozuk para: en az para sayısı *coin change (minimum number of coins)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Dinamik programlama (1B, en iyileme): enaz[t] = min(enaz[t − c] + 1)	Pozitif tamsayı paralar ve tutar; her paradan sınırsız	Θ(n·S)	Θ(n·S)	Θ(n·S)	O(S)

Özellikler	{1, 3, 4} ile 6: açgözlü 3 para, DP 2 para; kanonik sistemde ikisi aynı
Değişmez / doğruluk	t artarken her u < t için enaz[u] en az para sayısıdır (ya da ∞)
Kütüphane karşılığı	C: yok (elle) · C++: yok (elle) · Python: yok (elle)
Ne zaman kullanılır	Para sistemi kanonik değilse ya da bilinmiyorsa; en az adım problemleri
Ne zaman kullanılmaz	S çok büyükse (Θ(n·S): sözde polinom); kanonik sistemde açgözlü yeter

TAHMİN ET

Kaç farklı yolla ödenir? yol[t] = Σ yol[t – c], yol[0] = 1. Dış döngü PARA, iç döngü TUTAR iken {1, 2, 5} ile yol[5] = 4 (kombinasyon). Döngüler yer değiştirirse (dış

A 4: sıra fark etmez

B 5

C 9

D 10

Kaç farklı yolla ödenir? Aynı geçiş, farklı döngü sırası

Dış döngü PARA → kombinasyon

t	0	1	2	3	4	5
başta	1	0	0	0	0	0
+ para 1	1	**1**	**1**	**1**	**1**	**1**
+ para 2	1	1	**2**	**2**	**3**	**3**
+ para 5	1	1	2	2	3	**4**

yol[5] = 4: 5, 2+2+1, 2+1+1+1, 1+1+1+1+1

her ödeme bir kez (sıra önemsiz)

Dış döngü TUTAR → permütasyon

t	Σ yol[t − c] ← c	yol
1	1 ← c: 1	**1**
2	1 + 1 ← c: 1,2	**2**
3	2 + 1 ← c: 1,2	**3**
4	3 + 2 ← c: 1,2	**5**
5	5 + 3 + 1 ← c: 1,2,5	**9**

yol[5] = 9: 1+2 ile 2+1 ayrı sayılır

her paranın son adım olabildiği diziler

{1, 2, 5}, tutar 5: dış döngü para → kombinasyon (4); dış döngü tutar → permütasyon (9)

Üç dilde: kaç farklı yol? (dış döngü para)

H13_04_bozuk_para_yollar.c — C

```c
for (int i = 0; i < k; i++) {          /* DIŞ: para */
    int c = paralar[i];
    for (int t = c; t <= S; t++) {     /* İÇ: tutar */
        yol[t] = (yol[t] + yol[t - c]) % MOD;
    }
    if (satirlari_yaz) {
        printf("  para %d sonrası:", c);
        yaz_satir(S);
    }
}
```

H13_04_bozuk_para_yollar.cpp — C++

```cpp
for (int c : paralar) {                // DIŞ: para
    for (int t = c; t <= S; t++) {     // İÇ: tutar
        yol[t] = (yol[t] + yol[t - c]) % MOD;
    }
    if (satirlari_yaz) {
        std::cout << "  para " << c << " sonrası:";
        yaz_satir(S);
    }
}
```

H13_04_bozuk_para_yollar.py — Python

```python
for c in paralar:                  # DIŞ: para
    for t in range(c, S + 1):      # İÇ: tutar
        yol[t] = (yol[t] + yol[t - c]) % MOD
    if satirlari_yaz:
        print(f"  para {c} sonrası:" + yaz_satir(S))
```

Sayılar hızla büyür: üç dilde her toplamadan sonra mod $10^9 + 7$ (Python tamsayısı taşmaz ama çıktı aynı kalsın diye) (satırlar: C 44–53 · C++ 46–54 · Python 33–37)

2B DP: ızgarada yol sayısı ve en küçük maliyetli yol

Yol sayısı (3 × 4)

1	1	1	1
1	2	3	4
1	3	6	**10**

yol[i][j] = yol[i−1][j]
+ yol[i][j−1]

kenarlar 1 (yeşil)

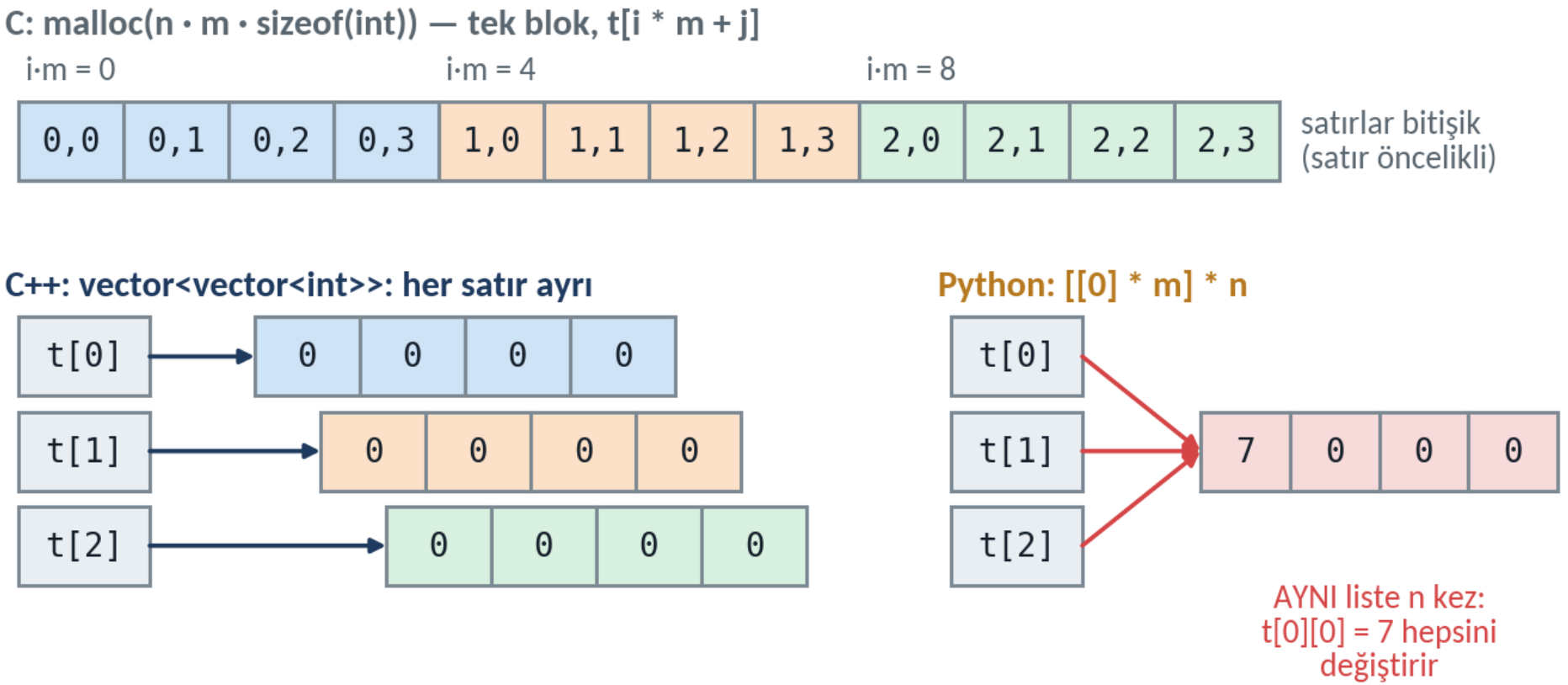
Engelli ızgarada en küçük maliyet (m = 15; # engel)

1	**1**	3	**4**	1	**5**	2	**7**	8	**15**	1	**16**
1	**2**	5	**7**	#		1	**8**	1	**9**	2	**11**
4	**6**	2	**8**	1	**9**	#		3	**12**	1	**12**
#		1	**9**	9	**18**	1	**19**	1	**13**	2	**14**
2	**∞**	1	**10**	2	**12**	4	**16**	#		1	**15**

hücrede: maliyet a (gri) · m[i][j] (kalın); kırmızı: geri çıkarılan yol

yol[2][3] = 10 = C(5, 2) · engelli 5 × 6: 13 yol (engelsiz 126) · en küçük maliyet 15 (eşitlikte yukarıdan)

2B tabloyu kurmak: tek blok, satır vektörleri, Python tuzağı



Doğrusu (Python): [[0] * m for _ in range(n)] — her satır yeni liste

*Python [[0] * m] * n: t[0][0] = 7 → ilk hücreler 7 7 7 7 7; Pascal C(5, 2) = 13 (doğrusu 10)*

En uzun artan alt dizi: uz[i] ve ebeveyn[i]

LIS $O(n^2)$: uz[i] = 1 + max{uz[j] : j < i, a[j] < a[i]}; ebeveyn zinciri

i	0	1	2	3	4	5	6	7
a[i]	5	**2**	8	6	**3**	**6**	**9**	7
uz[i]	1	**1**	2	2	**2**	**3**	**4**	4
ebeveyn	-1	**-1**	0	0	**1**	**4**	**5**	5



en uzun: uz = 4, son indis 6; zincir 6 → 5 → 4 → 1; alt dizi 2 3 6 9 (kırmızı: geri çıkarılan zincir)

a = {5, 2, 8, 6, 3, 6, 9, 7}: uz 1 1 2 2 2 3 4 4 · zincir 1 → 4 → 5 → 6 → 2 3 6 9

LIS O(n^2): iç döngü ve ebeveyn

```
algoritma LIS(a[0..n-1])
  i ← 0'dan n-1'e:  uz[i] ← 1;  ebeveyn[i] ← -1
    j ← 0'dan i-1'e:
      eğer a[j] < a[i] ve uz[j] + 1 > uz[i] ise
        uz[i] ← uz[j] + 1;  ebeveyn[i] ← j
  döndür en büyük uz[i]'nin ilk indisi   ▷ zincir: ebeveyn[]
```

c/H13_07_lis_n2.c · satır 31–37

C

```c
for (int j = 0; j < i; j++) {
    karsilastirma++;
    if (a[j] < a[i] && uz[j] + 1 > uz[i]) {
        uz[i] = uz[j] + 1;
        ebeveyn[i] = j;
    }
}
```

İç döngü n(n − 1)/2 kez: n = 8'de 28 karşılaştırma · n = 16 rastgele dizide 2^{16} alt kümeyi tarayan kâhin de 7 bulur

ALGORİTMA KİMLİK KARTI

En uzun artan alt dizi (LIS) *longest increasing subsequence*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
DP (1B): uz[i] = 1 + max uz[j] (j < i, a[j] < a[i]); ebeveyn dizisi	Karşılaştırılabilir öğeler; “artan” = KESİN artan (a[j] < a[i])	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n)$

Özellikler	a = {5, 2, 8, 6, 3, 6, 9, 7}: uzunluk 4; n = 16 rastgele dizide 2^{16} alt kümeli kâhinle aynı
Değişmez / doğruluk	i. adımdan sonra her k ≤ i için uz[k] = a[k] ile biten en uzun artanın boyu
Kütüphane karşılığı	C: yok (elle) · C++: yok (O(n log n)'de std::lower_bound) · Python: yok (O(n log n)'de bisect)
Ne zaman kullanılır	Sırası korunan en uzun zincir: sürüm dizileri, kutu iç içe geçirme
Ne zaman kullanılmaz	n büyükse $O(n^2)$ sürüm (n = 10^5 → $5 \cdot 10^9$ karşılaştırma): O(n log n) sürüm (Örnek 13)

TAHMİN ET

LCS: L[i][j], a'nın ilk i ve b'nin ilk j harfinin en uzun ortak alt dizisinin uzunluğudur. a = “ABCBDAB”, b = “BDCABA”. L[4][5] (“ABCB” ile “BDCAB”)

A 1

B 2

C 4

D 3

LCS: tablo doldurma (C'de tek blok, satır öncelikli)

```
algoritma LCS(a[0..n-1], b[0..m-1])   ▷ L[i][0] = L[0][j] = 0
  i ← 1'den n'ye, j ← 1'den m'ye (satır öncelikli):
    eğer a[i-1] = b[j-1] ise L[i][j] ← L[i-1][j-1] + 1
    değilse L[i][j] ← max(L[i-1][j], L[i][j-1])   ▷ eşitlikte yukarı
  döndür L[n][m]
```

c/H13_08_lcs.c · satır 36–44

C

```c
for (int j = 1; j <= m; j++) {
    if (a[i - 1] == b[j - 1]) {
        L[i * s + j] = L[(i - 1) * s + j - 1] + 1;
    } else if (L[(i - 1) * s + j] >= L[i * s + j - 1]) {
        L[i * s + j] = L[(i - 1) * s + j];
    } else {
        L[i * s + j] = L[i * s + j - 1];
    }
}
```

Θ(n·m) zaman ve bellek · satır düzeyinde diff: 4 ortak satır, 1 silinen, 2 eklenen satır (Örnek 8)

TAHMİN ET

Geri çıkarma (7, 6)'dan başlıyor: a[6] = B, b[5] = A ve L[6][6] = L[7][5] = 4. Sözde koddaki eşitlik kuralıyla ilk adım hangi hücreye gider?

A (6, 5): çapraz, B yazılır

B (6, 6): yukarı

C (7, 5): sol

D (5, 5): iki adım birden

En uzun ortak alt dizi: önek tablosu ve geri çıkarma

LCS: a = "ABCBDAB", b = "BDCABA" — L[7][6] = 4, ortak "BCBA"

	""	B	D	C	A	B	A
""	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	↖ 1	← 1	1	1	2	2
C	0	1	1	↖ 2	← 2	2	2
B	0	1	1	2	2	↖ 3	3
D	0	1	2	2	2	↑ 3	3
A	0	1	2	2	3	3	↖ 4
B	0	1	2	2	3	4	↑ 4

Geçiş:

a[i−1] = b[j−1] → L[i−1][j−1] + 1 (↖)

değilse max(L[i−1][j] ↑, L[i][j−1] ←)

eşitlikte önce yukarı (↑)

Doldurma: satır öncelikli (i, sonra j); her hücre sol, üst ve çapraz komşusundan SONRA (mor oklar: L[5][3]'ün bağımlıları).

Kırmızı: (7, 6)'dan başa geri çıkarma; ↖ hücrelerinin harfleri sondan başa: "BCBA".

"ABCBDAB" / "BDCABA": L[7][6] = 4, geri çıkarılan "BCBA" · mor: bir hücrenin üç bağımlısı · kırmızı: geri çıkarma yolu

Düzenleme (Levenshtein) uzaklığı: sil, ekle, değiştir

Düzenleme uzaklığı: "kitten" → "sitting" = D[6][7] = 3

	""	s	i	t	t	i	n	g
""	**0**	1	2	3	4	5	6	7
k	1	**1**	2	3	4	5	6	7
i	2	2	**1**	2	3	4	5	6
t	3	3	2	**1**	2	3	4	5
t	4	4	3	2	**1**	2	3	4
e	5	5	4	3	2	**2**	3	4
n	6	6	5	4	3	3	**2** →	**3**

Geçiş (en küçüğü):

D[i−1][j] + 1 (silme ↓)

D[i][j−1] + 1 (ekleme →)

D[i−1][j−1] + [a ≠ b] (değiştir/eşleş ↘)

temel: D[i][0] = i, D[0][j] = j

Geri çıkarılan işlemler (baştan sona):

değiştir k→s	değiştir e→i
eşleş i	eşleş n
eşleş t	ekle g
eşleş t	

eşitlikte: eşleşme > silme > ekleme > değiştirme

"kitten" → "sitting" = 3: değiştir k→s, eşleş i, eşleş t, eşleş t, değiştir e→i, eşleş n, ekle g

“Bunu mu demek istediniz?”: uzaklık tablosu ve en yakın 3 sözcük

c/H13_09_duzenleme_uzakligi.c · satır 75–83

C

```c
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            int sil = D[(i - 1) * s + j] + 1;
            int ekle = D[i * s + j - 1] + 1;
            int degis = D[(i - 1) * s + j - 1] + (a[i - 1] != b[j - 1]);
            int enk = sil < ekle ? sil : ekle;
            D[i * s + j] = degis < enk ? degis : enk;
        }
    }
```

ÖNERİLER · SÖZLÜK 3 215 SÖZCÜK · KOD NOKTASI DÜZEYİNDE

```
gunduz -> gündüz (2), buldu (3), bulduk (3) | bayt düzeyinde 1.: buldu (3)
sehre -> şehre (1), kere (2), sedire (2) | bayt düzeyinde 1.: kere (2)
kılç -> kılıç (1), hıç (2), kal (2) | bayt düzeyinde 1.: hıç (2)
padisah -> padişah (1), padişahla (3), padişahım (3) | bayt düzeyinde 1.: padişah (2)
esir -> sözlükte VAR | bayt düzeyinde 1.: esir (0)
```

ALGORİTMA KİMLİK KARTI

LCS ve düzenleme (Levenshtein) uzaklığı *LCS / edit distance*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
DP (2B, önek çiftleri); satır öncelikli; tablodan geri çıkarma	İki dizi (karakter, satır, kod noktası); eşitlikte yön sabit	**Θ(n·m)**	**Θ(n·m)**	**Θ(n·m)**	**O(n·m) / O(min(n, m))**

Özellikler	"BCBA" (4); kitten → sitting 3; iki satırla bellek O(min(n, m)), ama geri çıkarma kaybolur
Değişmez / doğruluk	L[i][j] (D[i][j]) = a[0..i), b[0..j) öneklerinin LCS'si (uzaklığı)
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (`difflib` LCS/Levenshtein değil)
Ne zaman kullanılır	diff, yazım düzeltme, DNA dizisi hizalama
Ne zaman kullanılmaz	Çok uzun dizilerde tam tablo (yalnız değer gerekiyorsa iki satır)

SÖZDE KOD → C

0/1 sırt çantası: al ya da alma

```
algoritma SIRT-ÇANTASI(a[1..n], d[1..n], W)   ▷ K[0][w] = 0
  i ← 1'den n'ye, w ← 0'dan W'ye:
    K[i][w] ← K[i−1][w]                                  ▷ alma
    eğer a[i] ≤ w ise K[i][w] ← max(K[i][w], K[i−1][w − a[i]] + d[i])   ▷ al
  döndür K[n][W]
```

c/H13_10_sirt_cantasi.c · satır 40–48

C

```c
int al_ma = K[(i - 1) * s + w];
int en_iyi = al_ma;
if (a[i - 1] <= w) {
    int al = K[(i - 1) * s + w - a[i - 1]] + d[i - 1];
    if (al > en_iyi) {
        en_iyi = al;
    }
}
K[i * s + w] = en_iyi;
```

12 ders, sınır 30 AKTS: DP 117 puan (29 AKTS), oran açgözlüsü 111 puan

TAHMİN ET

Nesneler A(5, 10) B(4, 40) C(6, 30) D(3, 50), W = 10. Satır C (ağırlık 6, değer 30) doldurulurken K[2][10] = 50, K[2][4] = 40. K[3][10] kaçtır?

A 50: C alınmaz

B 60

C 70: C alınır

D 90

0/1 sırt çantası: nesne × kapasite tablosu

0/1 sırt çantası: K[i][w] (nesne × kapasite), W = 10 → K[4][10] = 90

	0	1	2	3	4	5	6	7	8	9	10
i = 0	0	0	0	**0**	0	0	0	0	0	0	0
1 A(5,10)	0	0	0	**0**	0	10	10	10	10	10	10
2 B(4,40)	0	0	0	0	40	40	40	**40**	40	50	50
3 C(6,30)	0	0	0	0	40	40	40	**40**	40	50	70
4 D(3,50)	0	0	0	50	50	50	50	90	90	90	**90**

geçiş: K[i][w] = max(K[i−1][w], K[i−1][w − a_i] + d_i); yeşil: "al" seçeneği kazandı

kırmızı: geri çıkarma — D alındı (w: 10 → 7), C alınmadı, B alındı (7 → 3), A alınmadı

K[i][w] = K[i−1][w] ise nesne ALINMADI sayılır (eşitlik kuralı)

A(5, 10) B(4, 40) C(6, 30) D(3, 50), W = 10: K[4][10] = 90; geri çıkarma D+ C− B+ A− → {B, D}

TAHMİN ET

Tek satırlı 0/1 sırt çantası: dp[w] = max(dp[w], dp[w – a_i] + d_i), kapasite TERS (w = W … a_i) dolaşılır. Kapasite ARTAN (w = a_i … W) dolaşılırsa ne olur?

A Nesne birden çok kez alınır: dp[10] = 150 (sınırsız sırt çantası)

B Hiçbir şey: dp[10] yine 90

C Dizi sınırı aşılır: tanımsız davranış

D Sonuç küçülür: dp[10] = 70

Tek satıra inmek: kapasite neden TERS dolaşılır?

TERS: w = W, …, a_i — doğru 0/1

w	0	1	2	3	4	5	6	7	8	9	10
+A	0	0	0	0	0	10	10	10	10	10	10
+B	0	0	0	0	40	40	40	40	40	50	50
+C	0	0	0	0	40	40	40	40	40	50	70
+D	0	0	0	50	50	50	50	90	90	90	90

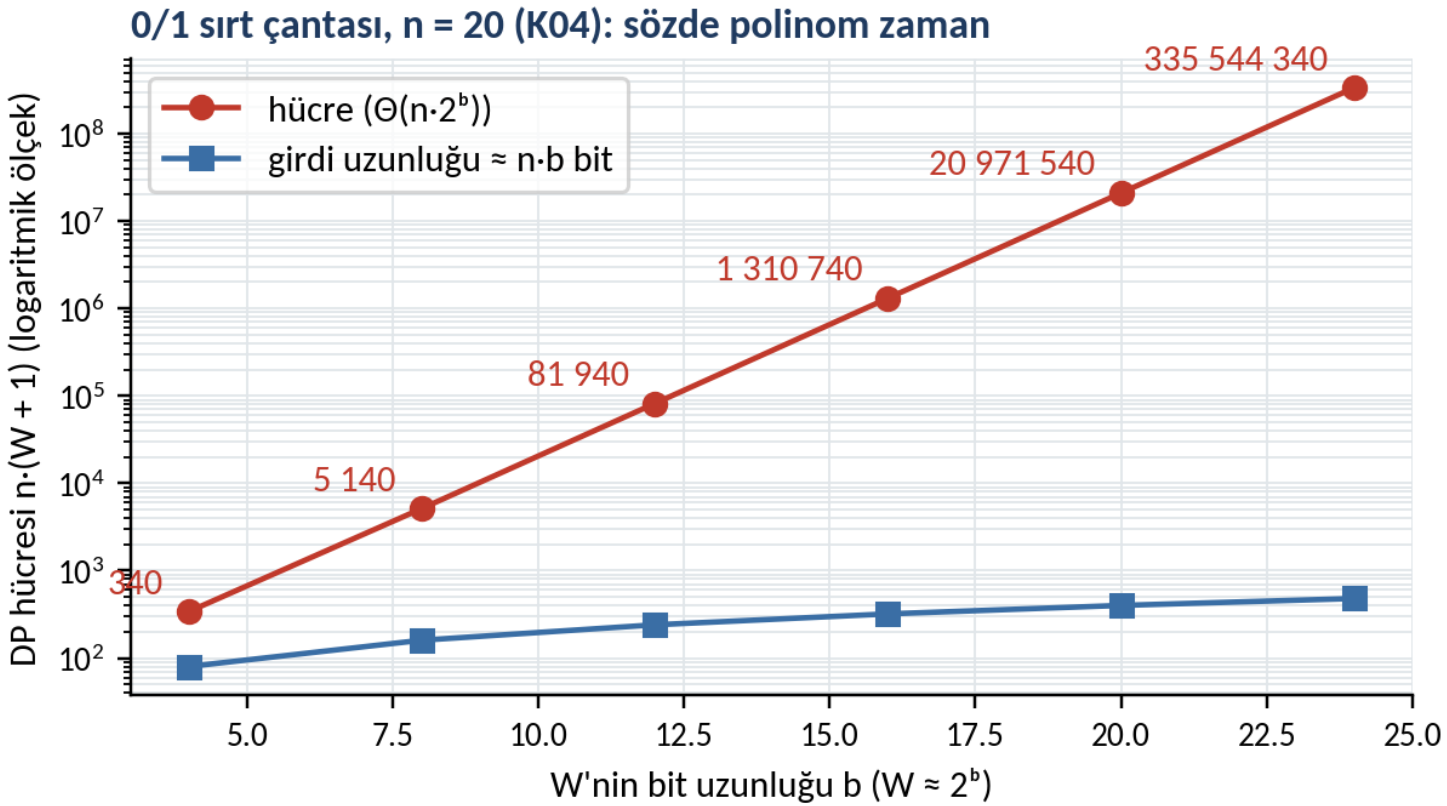
D(3, 50), ters: dp[w − 3] hâlâ önceki satırın değeri; dp[10]: 70 → 90

ARTAN: w = a_i, …, W — sınırsız (YANLIŞ)

w	0	1	2	3	4	5	6	7	8	9	10
+A	0	0	0	0	0	10	10	10	10	10	20
+B	0	0	0	0	40	40	40	40	80	80	80
+C	0	0	0	0	40	40	40	40	80	80	80
+D	0	0	0	50	50	50	100	100	100	150	150

D(3, 50), artan: dp[3] = 50 olur, sonra dp[6] = dp[3] + 50 = 100 (D iki kez); dp[9] = 150 (üç)

Sözde polinom zaman: Θ(n·W), ama W bir sayının değeri



- W'yi yazmak **b ≈ log₂ W** bit ister: W ≈ 2^b.
- Θ(n·W) = Θ(n·2^b): **bit uzunluğunda üstel**.
- Kavram 4: b 8 → 16 (girdi 2 kat), hücre 5 140 → 1 310 740 (**255,0 kat**).
- AKTS (W ≤ 1 000) gibi küçük sınırlarda DP rahat; W = 10^9'da değil.
- 0/1 sırt çantasının polinom zamanlı algoritması bilinmiyor: **NP-zorluk** (H14).

ALGORİTMA KİMLİK KARTI

0/1 sırt çantası *0/1 knapsack*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
DP (2B, nesne × kapasite); tek satırda kapasite TERS sırada	Tamsayı ağırlıklar ve kapasite W; her nesne en çok bir kez	Θ(n·W)	Θ(n·W)	Θ(n·W)	O(n·W) / O(W)

Özellikler	W = 10: 90 ({B, D}); oran açgözlüsü 160 ↔ DP 220; artan dolaşma 150 (yanlış)
Değişmez / doğruluk	i. satır dolunca K[i][w] = ilk i nesnenin w'ye sığan en iyi değeri
Kütüphane karşılığı	C: yok (elle) · C++: yok (elle) · Python: yok (elle)
Ne zaman kullanılır	Kapasite altında seçim (AKTS sınırıyla ders seçimi, v2.12); W küçükse
Ne zaman kullanılmaz	W çok büyükse (bit uzunluğunda üstel); kesirli sürümde açgözlü yeter (H12)

Alt küme toplamı: H3'ün ağacı ↔ DP tablosu

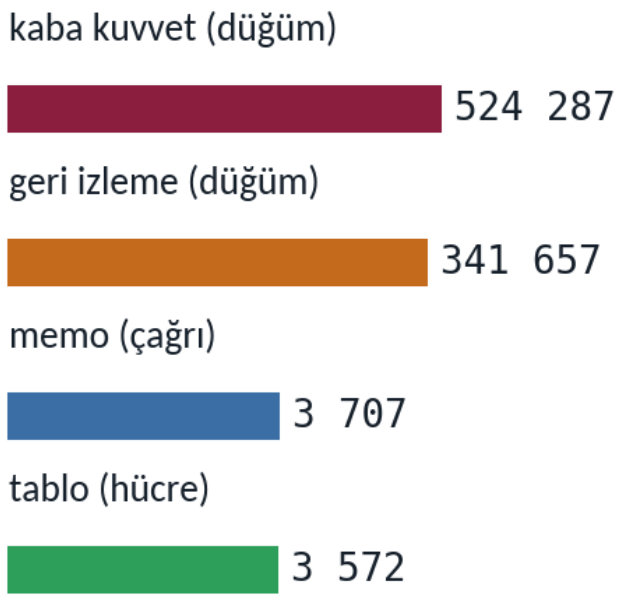
var[i][t]: a = {5, 4, 3, 2}, hedef 7

	0	1	2	3	4	5	6	7
i = 0	1	0	0	0	0	0	0	0
1 (5)	1	0	0	0	0	1	0	0
2 (4)	1	0	0	0	1	1	0	0
3 (3)	1	0	0	1	1	1	0	1
4 (2)	1	0	1	1	1	1	1	1

1 (yeşil): ilk i sayıyla t yapılabilir; say[4][7] = 2 alt küme

var[i][t] = var[i−1][t] VEYA var[i−1][t − a_i]

n = 18 (Örnek 12'nin girdisi)



çubuk uzunluğu: logaritmik

n = 18 (tohum 42): kaba 524 287 düğüm, geri izleme 341 657, memo 3 707 çağrı, tablo 3 572 hücre

Strateji karşılaştırma matrisi

Aynı problem, dört strateji: alt küme toplamı (n = 18 sayaçları; n = 26 süreleri, C, bu makinede)

Strateji	Doğruluk	Zaman (iş)	Bellek
Kaba kuvvet (H3)	her zaman doğru	Θ(2ⁿ): 524 287 düğüm; 584,7 ms	O(n) yığın
Budamalı geri izleme (H3)	her zaman doğru	en kötü O(2ⁿ): 341 657 düğüm; 386,6 ms	O(n) yığın
Açgözlü (H12)	YANLIŞ olabilir: %72,1 doğru*	O(n log n) (sıralama)	O(1) ek
DP (H13)	her zaman doğru	Θ(n·T): 3 572 hücre; 0,0057 ms	O(n·T); tek satırla O(T)

* n = 10…40, ulaşılabilir 2 027 hedefin 1 461 tanesinde tam tutturdu (büyükten küçüğe sığanı al)

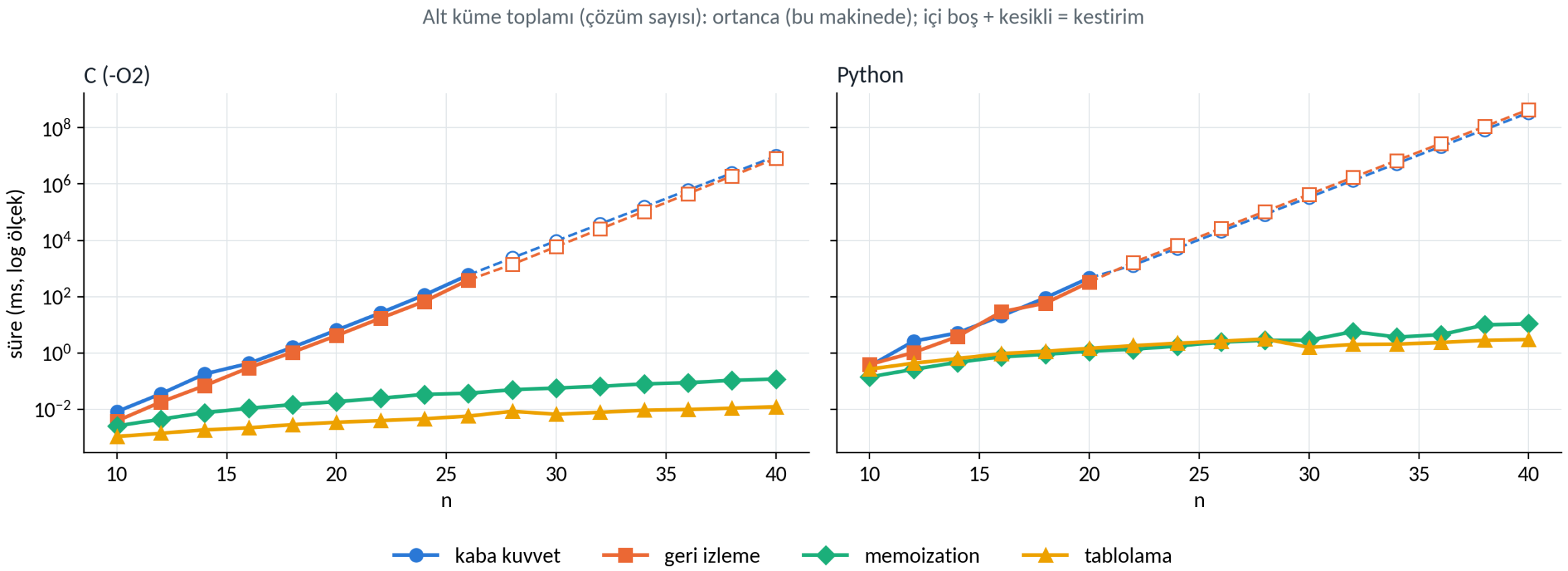
Açgözlü (büyükten küçüğe) ulaşılabilir 2 027 hedefin 1 461'ini tutturdu (%72,1)

KAPUTUN ALTINDA

Kaputun Altında: tablo bellekte nasıl durur?

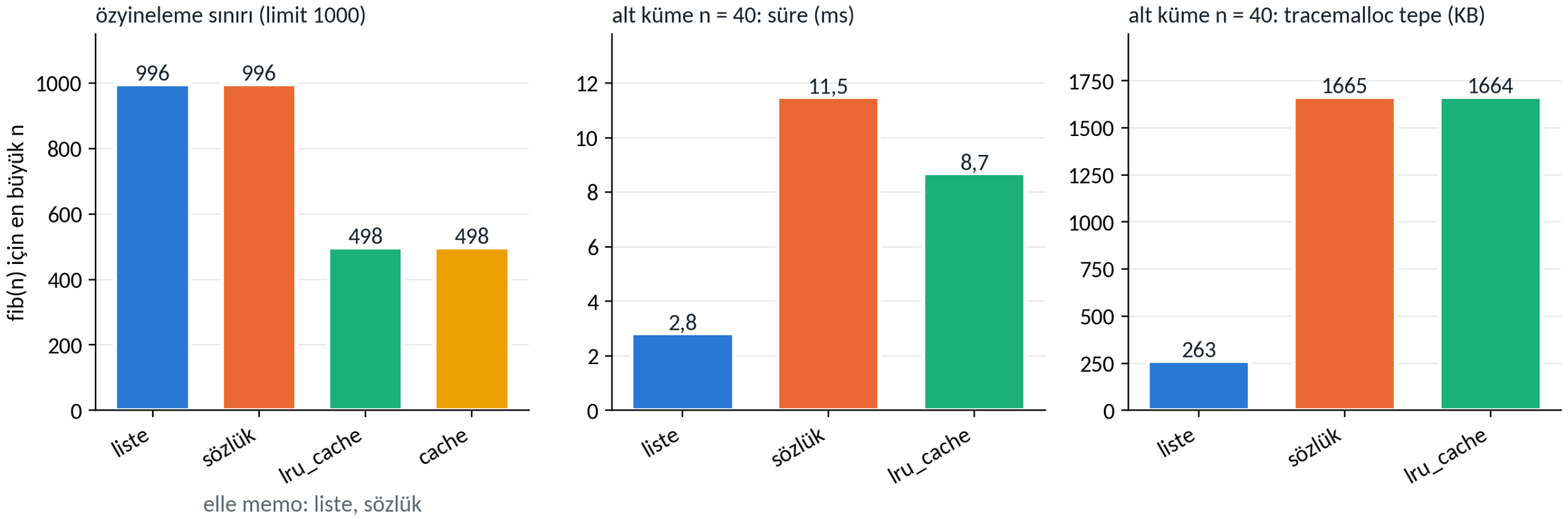
- **Satır öncelikli yerleşim:** L[i][j] ile L[i][j + 1] komşu. LCS'yi sütun sütun doldurmak C -O2'de en çok **6,15 kat** yavaş (n = 3 000); tablo ≈ 1 MB iken fark yok (1,07).
- **İki satıra inmek:** n = m = 20 000'de LCS'nin değerini iki satır (160 KB) bulur; tam tablo 1,60 GB gerektirirdi (Kavram 1). Bedeli: önceki satırlar silindi, **geri çıkarma kaybolur** (çare: Hirschberg, 1975).
- **Memo ve özyineleme derinliği:** sınır 1000 iken soğuk önbellekle elle memo n = 996'ya, `lru_cache`/`cache` n = 498'e kadar çalıştı (CPython 3.11 gözlemi). Çare: tablolama.

Haftanın Deneyi (a): alt küme toplamında dört strateji



C, n = 26: kaba 584,7 ms, tablo 0,0057 ms (102 680 kat); içi boş + kesikli: kestirim

Haftanın Deneyi (b): Python'da lru_cache ↔ elle yazılmış not



Python, alt küme n = 40: lru_cache elle listeye göre 3,07 kat süre, 6,32 kat bellek; özyineleme sınırı 498 ↔ 996

Hata Avcısı: altı sessiz hata

#	Hata	Belirti (bu makinede)	Çözüm
1	Temel durum: hepsi 0 ya da enaz[0] = ∞	6 için 0 para; ya da “ulaşılamaz”	enaz[0] = 0, ötekiler ∞
2	Doldurma sırası (matris zinciri, Python)	m[1][6] = 9000 (doğrusu 15125)	zincir uzunluğuna göre
3	Döngü sırası (sayım, C++)	{1, 2, 5}, 5: 9 küme (doğrusu 4)	dış döngü para
4	Tek satırda artan dolaşma (C)	dp[10] = 150 (doğrusu 90)	w = W … a_i (ters)
5	∞ = INT_MAX'a 1 eklemek (C)	enaz[5] = −2147483648 (-O0 ≠ -O2)	∞ denetimi toplamadan önce
6	Memo anahtarı yalnız i (Python)	en büyük değer 130 (doğrusu 90)	anahtar (i, w)

Hata Avcısı 5: ∞ + 1 taşar (tanımsız davranış)

hatalı: ∞ + 1

hata_avcisi/H13_HA5_sonsuz_tasmasi.c · satır 25–30 — C

```c
for (int i = 0; i < 2; i++) {
    int c = paralar[i];
    if (c <= t && enaz[t - c] + 1 < enaz[t]) {  /* HATA: ∞ + 1 */
        enaz[t] = enaz[t - c] + 1;
    }
}
```

düzeltilmiş

hata_avcisi/H13_HA5_sonsuz_tasmasi_duzeltilmis.c · satır 23–29 — C

```c
for (int i = 0; i < 2; i++) {
    int c = paralar[i];
    if (c <= t && enaz[t - c] != SONSUZ &&
        enaz[t - c] + 1 < enaz[t]) {
        enaz[t] = enaz[t - c] + 1;
    }
}
```

-O0: `enaz[ 6] = -2147483648` · -O2: `enaz[ 6] = 1` · düzeltilmiş: `enaz[ 5] = ∞`, `enaz[ 6] = 1`

Dil Penceresi: not tutma, tablolar ve büyük sayılar

	C	C++	Python
Not tablosu	dizi + −1 (hesaplanmadı); seyrekse karma	vector (−1) / unordered_map	liste, dict, functools.cache
2B tablo	malloc tek blok, t[i·m + j]	vector<vector<int>> (satırlar kopya)	[[0] * m for _ in range(n)]; tuzak: [[0] * m] * n
∞	INT_MAX + denetim	numeric_limits<int>::max()	math.inf ya da büyük int
Büyük sayılar	uint64_t sarar: F(94) = 1293530…; sayımda mod $10^9 + 7$	aynı	tam sonuç; mod isteğe bağlı
Özyineleme derinliği	yığın taşması: çökme	aynı	RecursionError (sınır 1000)

Mühendis Gözüyle · Atölyeler · İleri Bakış · Merak

Mühendis Gözüyle

- Bellek–zaman ödünleşimi: önce tablonun boyutunu hesaplayın (LCS 20 000²: 1,6 GB).
- Yalnız değer → satır tamponu; çözüm gerekiyorsa tam tablo ya da Hirschberg.
- v2.12: tam tablo + tek satır birlikte, değerleri karşılaştırılır.

Atölyeler

- DP reçetesi kartları: altı problem, kod yazmadan.
- Yazım düzeltici: Türkçe kelime listesiyle “Bunu mu demek istediniz?”, budama.
- Strateji turnuvası: alt küme toplamı, n = 10…40; kaba kuvvet n ≤ 25, ötesi kestirim.

İleri Bakış

- H14: Held–Karp (bit maskesi DP), NP-zorluk.
- Needleman–Wunsch hizalama; Bellman denklemi; Bellman–Ford, Floyd–Warshall, Viterbi; ağaç DP.
- Meydan okuma: Örnek 14 kelime bölme (önek DP'si), Örnek 15 matris zinciri (aralık DP'si).

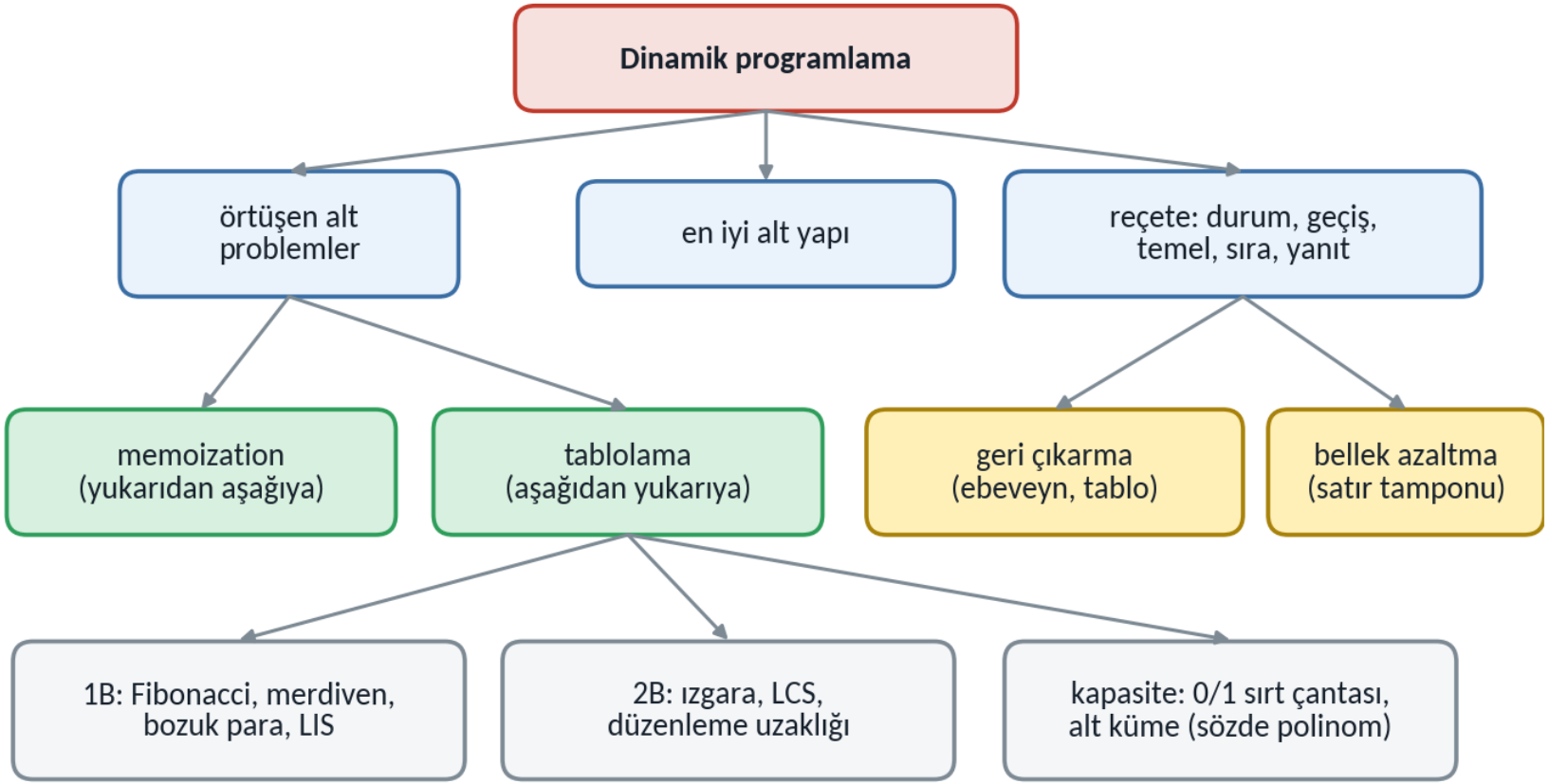
Merak: sabır sıralaması

- Örnek 13: LIS'i ikili aramayla O(n log n); deste sayısı = uzunluk.
- n = 10^5: 858 287 yoklama ↔ O(n^2) 4 999 950 000 karşılaştırma.
- Son tails dizisi neden bir alt dizi değildir?

Mini-OBS 2 v2.12: ders seçimi ve “Bunu mu demek istediniz?”

- **Ders seçimi = 0/1 sırt çantası:** AKTS sınırı W altında tercih puanı en büyük küme; tam tablo + geri çıkarma (eşitlikte ALMA), tek satır (c azalan).
- **Dört strateji (v2.12 belgesi § 8.1, W = 30):** v2.2'nin geri izleme ağacında düğüm n = 10 → 50: 896 → 147 198 469; DP hücresi 341 → 1 581.
- **Öneri:** ad, soyad ve tam ad terimlerinde **kod noktası** düzeyinde Levenshtein (ŞAHİN/SAHIN: 2, baytta 4); 100 000 kayıtta 300 000 terimden 1 679'u farklı.
- **Kabul:** 7 denetlenebilir ölçüt ve 13 test senaryosu (v2.12 belgesi § 10); `--hata-goster sinirsiz` artan dolaşmayı kâhine yakalatır.

Özet: dinamik programlama haritası



İki koşul → reçete → memo / tablo → geri çıkarma ve bellek azaltma → 1B, 2B ve kapasite boyutlu problemler

Hazırlık: bu hafta ve H14

- **Ders notu:** Bölüm 3–4 (reçete ve amiral algoritmalar), Bölüm 6 (deney), Bölüm 7 (Hata Avcısı).
- **Web sayfası:** Fibonacci çağrı ağacı, DP tablo doldurucu (LCS / düzenleme / sırt çantası), “Bunu mu demek istediniz?” ve strateji grafiği; haftalık quiz.
- **Alıştırmalar:** 13.1–13.10 bu hafta; 13.11–13.16 ve Mini-OBS 2 v2.12 gelecek derse.
- **H14 için:** sözde polinom zaman (Bölüm 4.6.2) ve Held–Karp'ın durum tanımı: “ziyaret edilen şehir kümesi + son şehir”.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Paralar {1, 4, 5}, tutar 8: açgözlü kaç para verir, DP kaç para? DP'nin ödemesini secim[]'den geri çıkarın.

2. LCS'yi iki satırla hesaplayınca neden uzunluk bulunur ama ortak alt dizinin kendisi bulunamaz?

3. 0/1 sırt çantası, n = 50, W = 10^6: tablo kaç hücredir? W'yi yazmak kaç bit ister; bu süre neden “sözde polinom”dur?